PERFORMANCE STUDY

# EJBCA Load Characteristics with a Lightweight Enclave HSM

*Benchmarking EJBCA's true issuance ceiling without CloudHSM cost, latency, or connection limits*

NitroEnclave · elastic enclave signer

Measured on AWS · us-east-1 · 0 errors

## 1 Thesis

Use the elastic enclave signer as a lightweight, high-throughput HSM to measure EJBCA's OWN load characteristics — without CloudHSM cost, latency, or connection limits distorting the result.

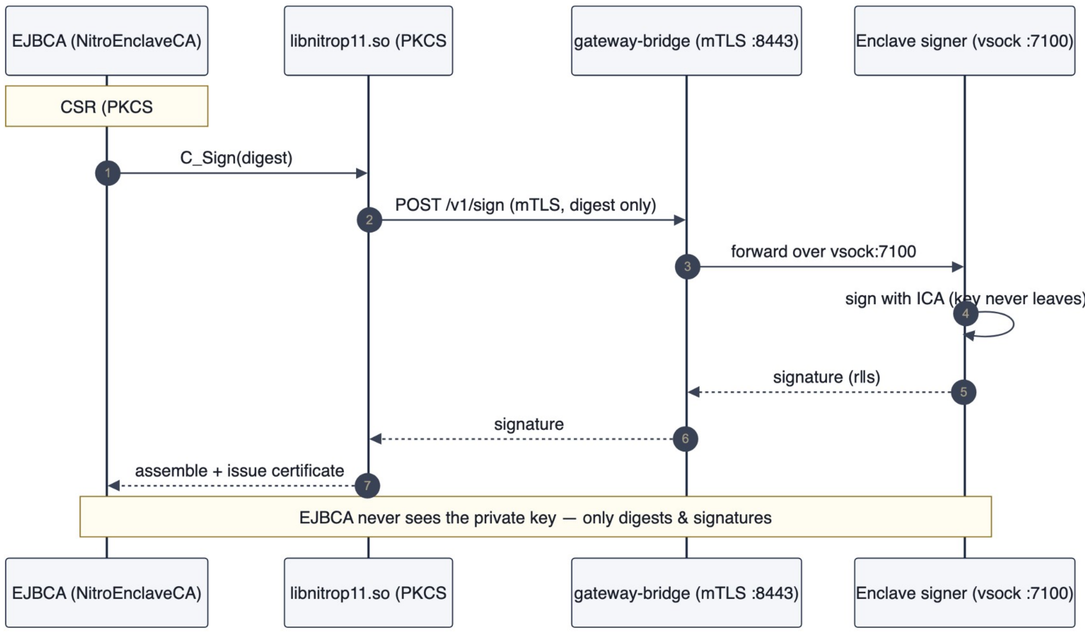
> **The signer is never the bottleneck**
>
> The enclave signer's raw ceiling (~8.6k signs/sec/box) sits far above EJBCA's issuance rate, so it never throttles the test. Whatever number you measure is EJBCA itself — not the HSM in front of it.

## 2 Why benchmark EJBCA this way

- **Removes the HSM as a variable —** The signer's raw ceiling (~8.6k/box) is far above EJBCA's issuance rate, so the HSM never throttles the test — you see EJBCA's real ceiling.
- **No CloudHSM complexity/cost —** No cluster to stand up, no per-HSM 24/7 bill, no connection cliff. Spin the enclave signer up on spot EC2, benchmark, tear down.
- **Full cipher coverage incl. PQC —** Benchmark RSA/ECDSA and ML-DSA/hybrid profiles that CloudHSM firmware cannot issue at all.

## 3 Method

We drove real PKCS#10 enrollments with cmd/enroll-loadtest against a single EJBCA CE node. The NitroEnclaveCA CryptoToken forwards every C_Sign over mTLS to the in-enclave ICA (ECDSA P-384), which signs the digest and returns the signature — EJBCA never sees the private key. Leaf certificates were ecdsa-p256; concurrency was swept across 16 / 32 / 64 clients at 2000 certs per run, with 0 errors on every run.




*Each enrollment: EJBCA → PKCS#11 .so → gateway → in-enclave sign → issued cert.*

## 4 Results

| Concurrency | certs/sec | note |
|---|---|---|
| **16** | 767 | warming |
| **32** | 895 | peak |
| **64** | 805 | saturated — p99 ~2× conc32 |

EJBCA peaked at ~895 certs/sec and then regressed past concurrency 32 (conc 64 fell to 805 with roughly double the p99 latency). The bottleneck is EJBCA's own parse / assemble / DB-write pipeline — not the signer behind it.

## 5 Interpretation — the signer is not the limit

The raw enclave sign ceiling is **~8,600 signs/sec on a single box** (ecdsa-p384) — about **10× the EJBCA issuance rate**. EJBCA saturates first, by a wide margin, so the measured ~895 certs/sec is a clean read on EJBCA's own capacity with the HSM removed as a variable.

> **Exactly the number you want**
>
> This is exactly the number you want when sizing EJBCA — its own ceiling, with the HSM removed as a variable.

## 6 Bonus — full cipher coverage incl. PQC

The same signer benchmarks RSA and ECDSA alongside ML-DSA / hybrid profiles that CloudHSM firmware cannot issue at all — so a single elastic signer covers both classical and post-quantum sizing on one test rig.

| Cipher | signs/sec (enclave ceiling) |
|---|---|
| **ECDSA P-384** | 7530 |
| **ML-DSA-65 (PQC)** | 6261 |
| **ML-KEM-1024** | 8478 |
| **RSA-4096** | 497 |

## 7 Takeaways for an EJBCA deployment

- **Plan for EJBCA's ceiling, not the HSM's —** size each EJBCA node to ~800–900 certs/sec and scale horizontally — add nodes, not a bigger HSM.
- **The enclave HSM adds capability, not drag —** custody (FIPS 140-3 L1, key never leaves the enclave), native PQC, and spot-EC2 elasticity — without becoming the bottleneck.
- **No CloudHSM needed —** no cluster to stand up, no per-HSM 24/7 bill, no connection cliff — for the benchmark or for production.
- **One rig, honest numbers —** remove the HSM as a variable and you measure EJBCA truthfully; the same rig then sizes classical and post-quantum profiles side by side.